

Automated Reinforcement Learning: Complete Study Guide

CartPole-v1 Stability and Hyperparameter Sensitivity

Table of Contents

1. Introduction and Project Overview
 2. Fundamental Concepts and Background
 3. Detailed Slide-by-Slide Explanation
 4. Key Formulas and Mathematical Foundations
 5. Graph Analysis and Interpretation
 6. Practical Implications and Conclusions
-

1. Introduction and Project Overview

What is this project about?

This project investigates **Automated Reinforcement Learning (AutoRL)** by comparing two policy gradient algorithms—**REINFORCE** and **PPO (Proximal Policy Optimization)**—on a classic control problem called **CartPole-v1**[1].

The Core Problem: Reproducibility in Reinforcement Learning

Reinforcement Learning (RL) is notoriously fragile. Training the same agent twice with identical code but different random seeds can produce completely different learning outcomes. Some runs succeed, others fail or plateau. This raises a critical question: **Is this due to algorithm design flaws, or inherent randomness in RL training?**[1]

Research Questions

The project answers three key questions:

1. **Algorithm Comparison:** How do REINFORCE and PPO compare in learning speed, stability, and convergence?
2. **Hyperparameter Sensitivity:** How much do key hyperparameters (discount factor, learning rate, clip range) affect PPO's performance?
3. **AutoRL Necessity:** Do results prove that automated hyperparameter tuning is essential, not just optional?[1]

Key Insight: Why Shaded Regions Matter

Throughout the presentation, you'll see graphs with solid lines and shaded bands. These are **confidence intervals** that visualize uncertainty:

- **Solid line** = Mean performance across multiple random seeds
- **Shaded region** = ± 1 standard deviation band around the mean
- **Narrow band** = Reproducible, stable results (good)
- **Wide band** = High variance, seed-dependent results (bad)[1]

This visualization makes the concept of "reproducibility" concrete and quantitative. Instead of showing one lucky run, we show what to expect on average when using the same algorithm with different random initializations.

2. Fundamental Concepts and Background

2.1 Markov Decision Processes (MDPs)

The CartPole environment is formally modeled as an **MDP**, which consists of:

- **State space (S)**: The agent observes 4 values at each timestep
 - Cart position on the track
 - Cart velocity (how fast it's moving)
 - Pole angle from vertical
 - Pole angular velocity (how fast it's rotating)
- **Action space (A)**: The agent can choose 2 discrete actions
 - Action 0: Push cart left
 - Action 1: Push cart right
- **Transition dynamics (P)**: The physics of the environment
 - How states change based on actions (governed by CartPole physics)
- **Reward function (R)**: Immediate feedback
 - +1 for each timestep the pole remains balanced
 - 0 when the pole falls or cart goes out of bounds
- **Discount factor (γ)**: Controls how much future rewards matter
 - Values close to 1.0 (e.g., 0.99) = agent values long-term returns
 - Values close to 0 (e.g., 0.10) = agent focuses on immediate rewards[1]

2.2 Expected Return and the Objective

The fundamental goal in RL is to maximize the **expected discounted return**:

$$J(\theta) = E \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Where:

- θ = neural network weights (what we're trying to optimize)
- $E[\cdot]$ = expectation (average over all possible trajectories)
- γ = discount factor ($0 < \gamma \leq 1$)
- r_t = immediate reward at timestep t [1]

What this means in plain English: The agent wants to maximize the sum of future rewards, but rewards far in the future are discounted (weighted less). For CartPole, each balanced timestep gives reward 1, so the maximum possible return is 500 (max episode length).

2.3 Policy Gradient Methods: The Core Idea

Both REINFORCE and PPO are **policy gradient methods**. They work by:

1. **Running an episode** with the current policy
2. **Computing how much each action contributed** to the final return
3. **Updating weights to increase probability of good actions** and decrease probability of bad actions
4. **Repeating** until convergence[1]

The key insight is that we update the policy **in the direction of the gradient** that increases expected return. This is analogous to walking downhill on a landscape to find a minimum—here, we're climbing a hill to maximize return.

2.4 Why Multiple Seeds?

When we train a neural network with stochastic (random) gradient descent, multiple sources of randomness affect training:

1. **Weight initialization:** Random starting values for neural network parameters
2. **Batch sampling:** Which experiences are selected for training
3. **Exploration randomness:** Which actions the agent takes (partly random, partly learned)[1]

To handle this properly, the project follows best practices by:

- Running each experiment with multiple **random seeds** (3-5 different initializations)
- Computing **mean and standard deviation** across seeds for each timestep
- Visualizing results as mean $\pm$ std (shaded bands)[1]

This gives honest, reproducible results instead of cherry-picking lucky runs.

3. Detailed Slide-by-Slide Explanation

Slide 1: Title Slide

Content: Introduction to the project

- Title: "Automated Reinforcement Learning"
- Subtitle: "Stability & Hyperparameter Sensitivity on CartPole-v1"
- A Comparative Study of Policy Gradient Methods: REINFORCE vs PPO

What it establishes: The project scope—comparing two policy gradient algorithms on a concrete benchmark and investigating hyperparameter sensitivity.

Slide 2: Presentation Outline

Content: Seven main topics to be covered

1. **Problem Definition & Motivation:** Why this project matters
2. **System Architecture (CartPole Environment):** How the environment works
3. **REINFORCE Algorithm:** Theory and implementation
4. **PPO Algorithm:** Theory and implementation
5. **Algorithm Comparison:** Head-to-head results
6. **Hyperparameter Sensitivity Analysis:** How sensitive PPO is to key settings
7. **Conclusions & Future Directions:** Summary and next steps

Key Takeaway: This is a systematic study progressing from foundational concepts to comparative analysis to practical insights.

Slide 3: Problem Definition & Motivation

The Challenge:

"Balance a pole on a moving cart without it falling over"

This is a classic control problem in RL that has three important properties:

1. **Simple enough to understand** (unlike complex game environments)
2. **Complex enough to be meaningful** (not trivial)
3. **Fast to train** (episodes complete in milliseconds, so we can run many experiments)

Why Policy Gradient Methods?

- REINFORCE and PPO are **fundamental to modern RL**
- PPO is the **industry standard** used by OpenAI and DeepMind
- Understanding **hyperparameter sensitivity** directly improves real-world performance[1]

Research Motivation:

The project asks: If RL is so fragile that small changes break things, how can we trust it for critical applications? The answer is **AutoRL**—automatically tuning hyperparameters instead of manual trial-and-error.

Slide 4: System Architecture – RL Agent Loop

The Core Loop (repeats every timestep):

1. **Observe State:** Neural network sees 4 state values
 - Cart position, velocity, pole angle, angular velocity
2. **Choose Action:** Neural network outputs probabilities
 - $P(\text{Action } 0)$ and $P(\text{Action } 1)$
 - Agent randomly samples based on these probabilities
3. **Get Reward:** Environment returns feedback
 - +1 if pole balanced, episode continues
 - 0 if pole fell, episode ends
4. **Update Network:** Learning algorithm adjusts weights
 - REINFORCE and PPO use different update rules

- Goal: increase probability of actions that led to high returns[1]

The Goal: Maximize total reward over 500 steps (perfect balance = 500 reward)

Random baseline: An untrained agent that picks actions randomly gets ~20-25 reward.

Slide 5: CartPole-v1 Complete Environment Reference

State Space (4-dimensional):

- Cart position: $x \in [-2.4, 2.4]$
- Cart velocity: $\dot{x}$ (continuous)
- Pole angle: $\theta \in [-0.209, 0.209]$ radians $\approx \pm 12^\circ$
- Angular velocity: $\dot{\theta}$ (continuous)

Action Space (2 discrete actions):

- Action 0 = Push cart left with fixed force
- Action 1 = Push cart right with fixed force

Reward Signal:

- +1 for each timestep (pole balanced)
- Episode terminates if:
 - $x > 2.4$ or $x < -2.4$ (cart goes out of bounds)
 - $\theta > 0.209$ or $\theta < -0.209$ (pole falls too far)[1]

Success Criteria:

- Max episode length: 500 timesteps
 - Success threshold: average reward ≥ 475 over 100 episodes
 - Perfect score: 500 reward (balanced entire episode)[1]
-

Slide 6: REINFORCE Algorithm – Theory & Implementation

Core Principle

REINFORCE is the simplest policy gradient algorithm. The key idea:

"Increase the probability of actions that led to high returns, decrease those that led to low returns."

The REINFORCE Update Rule

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

Breaking down this formula[1]:

- $\nabla J(\theta)$ = Gradient of expected return (what we're computing)
- $\frac{1}{N}$ = Average over N episodes (batch of training data)
- $\sum_{i=1}^N$ = Sum over all N episodes in batch
- $\sum_{t=0}^{T_i}$ = Sum over all timesteps in episode i

- θ = Neural network parameters/weights
- ∇ = Gradient operator (direction of steepest increase)
- $\log \pi_{\theta}(a_t|s_t)$ = Log probability of action a_t in state s_t
- G_t = Cumulative discounted return from timestep t onward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Why log-probability?

The log of a probability makes the math work out nicely (via the policy gradient theorem).
When we multiply the gradient by G_t :

- If G_t is **large and positive**, we increase $\log \pi(a_t|s_t) \rightarrow$ increase $P(\text{action})$
- If G_t is **small or negative**, we decrease $\log \pi(a_t|s_t) \rightarrow$ decrease $P(\text{action})$ [1]

Why REINFORCE has High Variance

REINFORCE estimates the gradient using **only one trajectory per update**. This means:

1. **Random episodes cause fluctuation:** A lucky episode gives a large positive gradient; an unlucky episode gives a large negative gradient
2. **Return G_t is noisy:** The entire episode reward flows back through time, so one small mistake at the end can wipe out progress
3. **Result:** Learning curves are jagged, unstable, and hard to reproduce[1]

REINFORCE Characteristics

Pros:

- Conceptually simple and easy to understand
- Theoretically guaranteed to converge (unbiased gradient estimator)
- No bias—finds true policy gradient[1]

Cons:

- **Very high variance** requires enormous sample sizes
- **Slow learning** takes many more episodes than PPO
- **Unstable curves** with large jumps and drops
- **Poor reproducibility** across random seeds[1]

Implementation Details (from your project)[1]:

- Custom PyTorch implementation (not from library)
 - 2-layer neural network: input(4) $\rightarrow$ hidden(128) $\rightarrow$ hidden(128) $\rightarrow$ output(2)
 - Learning rate: $\alpha = 10^{-3}$ (relatively high)
 - Discount factor: $\gamma = 0.99$
 - Activation: ReLU (Rectified Linear Unit)
-

Slide 7: PPO Algorithm – Theory & Implementation

Core Innovation: Clipping Mechanism

PPO addresses REINFORCE's instability by introducing a **clipping mechanism** that prevents the policy from changing too drastically in one update.

The PPO Clipped Objective

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

This formula needs breaking down carefully[1]:

- L^{CLIP} = The loss function we want to maximize
- $E_t[\cdot]$ = Expected value (averaged over timesteps in batch)
- $\min(\cdot, \cdot)$ = Takes the minimum of two values
- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ = **Probability ratio**
 - Numerator: probability of action under NEW policy
 - Denominator: probability of action under OLD policy
 - Values close to 1 = policy hasn't changed much
 - Values far from 1 = policy changed drastically[1]
- $\hat{A}_t$ = **Advantage estimate** (generalized advantage estimation)
 - How much better is this action compared to the average?
 - If $\hat{A}_t > 0$: action better than average → should increase probability
 - If $\hat{A}_t < 0$: action worse than average → should decrease probability[1]
- $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ = **Clipping function**
 - Restricts r_t to the interval $[1 - \epsilon, 1 + \epsilon]$
 - With $\epsilon = 0.2$ (clip range), allows $r_t \in [0.8, 1.2]$
 - If r_t tries to go outside this range, it's clipped back in
 - This prevents the policy from changing too fast[1]

How the Clipping Works: The Minimum Trick

The inner **minimum** has clever interpretation:

Case 1: Advantage is positive ($\hat{A}_t > 0$, good action)

- We want to increase $\pi(a_t|s_t)$, so r_t should increase
- If $r_t < 1 + \epsilon$: unclipped term $r_t \hat{A}_t$ is smaller (and we use it)
- If $r_t > 1 + \epsilon$: clipped term $(1 + \epsilon) \hat{A}_t$ is smaller (and we use it instead)
- **Result:** Policy increases, but not beyond the safety zone

Case 2: Advantage is negative ($\hat{A}_t < 0$, bad action)

- We want to decrease $\pi(a_t|s_t)$, so r_t should decrease
- If $r_t > 1 - \epsilon$: unclipped term $r_t \hat{A}_t$ is less negative (and we use it)
- If $r_t < 1 - \epsilon$: clipped term $(1 - \epsilon) \hat{A}_t$ is less negative (and we use it)
- **Result:** Policy decreases, but not beyond the safety zone[1]

The Net Effect: The gradient only flows when it's safe and beneficial. This is PPO's "safety belt"—it prevents catastrophic policy collapse.

Why This Matters

- **Lower variance** than REINFORCE due to advantage estimation
- **Stable updates** because clipping prevents wild policy changes
- **Better sample efficiency** learns faster with fewer environment interactions
- **More reproducible** across different random seeds[1]

PPO Characteristics

Pros:

- Significantly more stable than REINFORCE
- Better sample efficiency (achieves goals in fewer steps)
- Clipping mechanism acts as safety valve
- State-of-the-art in practice (OpenAI, DeepMind standard)[1]

Cons:

- More sensitive to hyperparameters than REINFORCE
- Requires tuning: γ (discount factor), α (learning rate), ϵ (clip range)
- More complex theory than REINFORCE[1]

Implementation Details (from your project)[1]:

- From Stable-Baselines3 library (widely-used, battle-tested implementation)
- Follows original PPO paper closely
- 2-layer MLP: input(4) → hidden(128) → hidden(128) → output(2)
- Default hyperparameters initially:
 - Discount factor: $\gamma = 0.99$
 - Clip range: $\epsilon = 0.2$
 - Learning rate: $\alpha = 3 \times 10^{-4}$
 - Batch size: 64
- Training budget: 50,000 timesteps (roughly 300-500 episodes depending on learning speed)

Slide 8: Algorithm Comparison Results

Graph: Learning curves comparing three agents over ~250 episodes

The Three Curves:

1. **Random agent** (bottom): Stays flat at ~20-25 reward
 - Takes random actions, never learns
 - Baseline to show what untrained performance looks like[1]
2. **REINFORCE** (blue, jagged): Reaches high reward but takes longer
 - Reaches ~500 reward after ~200-250 episodes
 - **Shaded band is WIDE:** Large variance between seeds
 - **Curve is jagged:** Individual episodes show high and low rewards
 - Shows instability even after learning[1]
3. **PPO** (orange, smooth): Reaches high reward quickly and smoothly
 - Reaches ~500 reward after ~150 episodes
 - **Shaded band becomes NARROW:** After initial warm-up, very consistent
 - **Curve is smooth:** Steady upward trend with little fluctuation

- Shows strong reproducibility[1]

Key Observation:

"PPO converges 40-50% faster than REINFORCE and achieves higher rewards with better stability across random seeds."[1]

What the narrow vs. wide shaded band tells us:

- **Narrow band** (PPO) = If you run this algorithm again with a different random seed, you'll get similar results (reproducible)
- **Wide band** (REINFORCE) = If you run this algorithm again, you might get very different results (unpredictable)[1]

This is exactly what AutoRL solves—by automatically finding PPO's best hyperparameters, we get consistent, reproducible performance.

Slide 9: Experimental Setup & Configuration

Comprehensive details of how experiments were run[1]:

Component	Setting	Purpose
Environment	CartPole-v1 from OpenAI Gym	Standard benchmark
Training Budget	50,000 timesteps	Fair comparison (enough to learn)
Batch Size	64	Neural network update frequency
Network	2-layer MLP: 128 → 128 units	Standard architecture for CartPole
Base Learning Rate	3×10^{-4}	Default for PPO
Discount Factor	$\gamma = 0.99$	Balances immediate vs future rewards
Random Seeds	3-5	Ensures reproducibility analysis

Why these specific choices?

- **50,000 timesteps:** Long enough to solve CartPole (random agent gets 20-25, success is 475+), not so long that experiments take forever
- **Batch size 64:** Common default for PPO; balance between stability and computation
- **128 hidden units:** Sufficient for CartPole's simple 4D state space

- $\gamma = 0.99$: Tuned to be "good" (you'll see it's compared against 0.90 and 0.999 later)
- **Multiple seeds**: Follows Henderson et al. best practices for honest reporting[1]

Slide 10: PPO Stability Across Random Seeds

Graph: PPO trained with 5 different random seeds (0-4), same hyperparameters

What the graph shows[1]:

- **X-axis:** Episode number (0-300)
- **Y-axis:** Episode reward (0-500)
- **Solid line:** Mean reward across all 5 seeds
- **Shaded band:** ± 1 standard deviation region

Interpretation of results:

1. **Initial phase (episodes 0-50):** Shaded band is WIDE
 - Policy is random initially, different seeds explore differently
 - High variance is expected (agent is learning)
2. **Convergence phase (episodes 50-150):** Band narrows dramatically
 - All seeds reach similar performance levels
 - Policy has converged to a good solution
3. **Mature phase (episodes 150+):** Band is VERY NARROW
 - All seeds achieve 500 reward (perfect balance)
 - Minimal variation between seeds
 - Policy is robust and reproducible[1]

Why is this important?

"This is exactly the type of behavior we would like to see before placing an automated hyperparameter search on top."[1]

A stable algorithm (narrow bands after training) is a good candidate for AutoRL. If the algorithm were chaotic even with fixed hyperparameters, AutoRL would have nothing to optimize toward.

Slide 11: Discount Factor Sensitivity – Gamma (γ)

Graph: Three lines showing PPO learning with $\gamma \in \{0.90, 0.99, 0.999\}$

The three lines (each is mean $\pm$ std over 3 seeds):

1. $\gamma = 0.90$ (**blue**): Myopic Agent
 - Learns only to immediate next state, doesn't plan ahead
 - Curve reaches ~200 reward then **plateaus hard**
 - **Shaded band stays WIDE** (inconsistent across seeds)
 - Even though episode length is 500, agent only cares about next step
 - In CartPole, immediate decisions aren't enough[1]
2. $\gamma = 0.99$ (**orange**): OPTIMAL
 - Best balance between myopic and greedy behavior
 - **Reaches 500 reward quickly** (~150 episodes)
 - **Shaded band becomes very narrow** (consistent, reproducible)
 - "Best choice for this environment"

- Sets the bar for AutoRL to optimize toward[1]
- 3. $\gamma = 0.999$ (**green**): Greedy Agent
 - Overvalues distant future, too patient
 - Also solves the task but **takes longer** (~200+ episodes)
 - **Shaded band remains moderately wide** (more variability)
 - Learning signals are delayed (reward from episode end takes time to propagate backward)
 - Overvaluing distant future actually slows learning[1]

Mathematical Intuition:

Discount factor γ affects how much we care about future rewards:

- Returns become: $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$
- With $\gamma = 0.90$: $G_t = r_t + 0.90r_{t+1} + 0.81r_{t+2} + \dots$
 - r_{t+1} is 90% weighted, r_{t+2} is 81% weighted, etc.
 - After 10 steps: weight is ~0.35, essentially ignoring far future
- With $\gamma = 0.99$: $G_t = r_t + 0.99r_{t+1} + 0.98r_{t+2} + \dots$
 - After 100 steps: weight is still ~37%, values long horizons
 - Better for episodic tasks with length 500[1]
- With $\gamma = 0.999$: Weight decays even slower, but may be too patient[1]

Key Finding:

"Changing γ from 0.99 to 0.90 can prevent the algorithm from solving the environment, despite identical code and similar training budget."[1]

This is why AutoRL matters: Manual tuning would easily miss this sweet spot.

Slide 12: Clip Range Sensitivity – Epsilon (ϵ)

Graph: Three lines showing PPO learning with $\epsilon \in \{0.1, 0.2, 0.4\}$

Reminder: Clip range ϵ controls how much the policy probability ratio r_t can change:

- Ratio $r_t \in [1 - \epsilon, 1 + \epsilon]$ is allowed
- If r_t tries to go outside, it's clipped[1]

The three lines (each single seed for clarity):

1. $\epsilon = 0.1$ (**red**): Overly Restrictive
 - Policy updates are **too conservative**
 - Gradient barely flows even when beneficial
 - **Learning is very slow**: takes many episodes
 - Agent eventually reaches 500 but takes much longer than optimal
 - **Limited exploration**: Being too cautious prevents trying new things[1]
2. $\epsilon = 0.2$ (**orange**): OPTIMAL (Default)
 - **Sweet spot between caution and aggression**
 - Fast convergence (agent learns quickly)
 - **Smooth trajectory** (no sharp jumps)
 - Balanced clipping: prevents catastrophe but allows meaningful updates
 - Recommended by original PPO paper for good reason[1]
3. $\epsilon = 0.4$ (**green**): Overly Permissive

- Policy updates are **too aggressive**
- Changes probability ratio by up to 40% in one step
- **Reaches 500 very quickly** (overshoots)
- **Curve shows sharp oscillations**: policy overshoots then corrects
- Close to clipping "safety guarantee" limit
- Risk: In harder environments, could cause divergence[1]

Mathematical Intuition:

The clipping range controls the trust region. Think of it like:

- $\epsilon = 0.1$: Take tiny cautious steps, very safe but slow
- $\epsilon = 0.2$: Take medium steps, balanced safety and speed
- $\epsilon = 0.4$: Take large aggressive steps, fast but risky[1]

Key Finding:

"Default value 0.2 is a good compromise between speed and stability. Too small = slow; too large = oscillatory."[1]

Slide 13: Learning Rate Sensitivity – Alpha (α)

Graph: Three lines showing PPO learning with $\alpha \in \{3 \times 10^{-5}, 3 \times 10^{-4}, 3 \times 10^{-3}\}$

Reminder: Learning rate α controls step size in gradient descent

- Weights update: $\theta_{new} = \theta_{old} + \alpha \nabla J(\theta)$
- Larger α = larger steps = faster learning but risk overshooting
- Smaller α = smaller steps = stable but slow learning[1]

The three lines (single seed):

1. $\alpha = 3 \times 10^{-5}$ (**blue**): Too Small
 - **Extremely slow improvement** barely visible at start
 - Gradient updates are microscopic
 - **Does not reach 500 reward** within 50,000 timesteps
 - Agent is learning but at glacial pace
 - This rate might work if training for 10× longer[1]
2. $\alpha = 3 \times 10^{-4}$ (**orange**): OPTIMAL (Default)
 - **Best convergence rate**: learns quickly but not recklessly
 - Reaches 500 reward in ~150 episodes
 - **Smooth stable curve** shows balanced updates
 - Sweet spot: "good trade-off between speed and stability"[1]
3. $\alpha = 3 \times 10^{-3}$ (**green**): Too Large
 - **Very rapid initial learning** shoots up quickly
 - **Unstable after reaching peak**: curve shows oscillations
 - Overshoots, then undershoots, then corrects again
 - Updates are so large they destabilize the learned policy
 - In more complex problems, could diverge entirely[1]

Why These Specific Values?

These are spaced logarithmically on a scale:

- $3 \times 10^{-5} \rightarrow 3 \times 10^{-4}$ (10× increase)
- $3 \times 10^{-4} \rightarrow 3 \times 10^{-3}$ (10× increase)

This is standard practice for hyperparameter sweeps—exploring orders of magnitude differences.

Key Finding:

"For automated tuning, simply sampling learning rates on a wide log-scale can easily hit both too-slow and too-unstable regions." [1]

Slide 14: Multi-Seed Gamma Sensitivity – Robust Analysis

Graph: Gamma sensitivity ($\gamma \in \{0.90, 0.99, 0.999\}$) averaged over 3 seeds each

This is a more robust version of Slide 11—instead of single seeds, each curve now shows mean $\pm$ std over 3 different random initializations.

The three curves (each mean $\pm$ std over 3 seeds):

1. **$\gamma = 0.90$ (blue):**
 - Plateaus at ~200 with wide shaded band
 - Both the mean performance and variability are poor
 - Doesn't consistently solve the task
2. **$\gamma = 0.99$ (orange):**
 - **Tightest, fastest convergence** (shaded band is noticeably smallest)
 - Reaches 500 quickly and consistently
 - All 3 seeds converge to same high reward
 - Reproducibility winner
3. **$\gamma = 0.999$ (green):**
 - Eventually solves but slower than 0.99
 - Wider bands than 0.99, more seed-dependent
 - Overvaluing distant rewards delays learning signals [1]

Why show this again with multiple seeds?

Slide 11 was single seed (easier to read), but Slide 14 adds rigor by averaging over seeds. This prevents any one result from being luck. Both confirm: **$\gamma = 0.99$ is optimal for CartPole-v1.**

Slide 15: Training Stability – Mean & Std Comparison

Graph: Comparing PPO (green) vs REINFORCE (orange) with mean $\pm$ std bands

What this shows quantitatively:

The shaded regions directly visualize stability:

- **REINFORCE (orange):** Wide band throughout entire training
 - Even after 250 episodes, uncertainty is large
 - Indicates that random seed choice significantly affects outcome
 - Some seeds succeed, others plateau or fail
 - Difficult to rely on for production systems [1]
- **PPO (green):** Band narrows over time

- Starts wide (initial randomness) then narrows
- After ~100 episodes, band is tight
- All seeds converge to ~500 reward
- Indicates reproducible, reliable learning
- Good candidate for production systems[1]

Statistical Interpretation:

Standard deviation (σ) measures spread:

$$\sigma = \sqrt{\frac{1}{K} \sum_{k=1}^K (R_k^e - \bar{R}^e)^2}$$

Where:

- R_k^e = reward in episode e for seed k
- $\bar{R}^e$ = mean reward across K seeds
- Shaded region = $[\bar{R}^e - \sigma, \bar{R}^e + \sigma]$ (± 1 std)[1]

Why std deviation is used:

- Measures variability/uncertainty
- Larger σ = less predictable (bad for reliable systems)
- Smaller σ = more predictable (good for reliable systems)
- Non-zero σ is expected when using multiple seeds (randomness is real)[1]

Key Takeaway:

"PPO's standard-deviation bands become small after training, while REINFORCE keeps wide bands and large fluctuations. This quantitatively supports the statement that PPO is superior."[1]

Slide 16: Smoothed Performance Trends

Graph: Single REINFORCE run with raw (blue) and smoothed (orange) curves

The two lines:

1. **Raw curve (blue):** Episode-by-episode rewards
 - Extremely jagged and noisy
 - Individual episode outcomes vary widely
 - Hard to see underlying trend
 - Shows REINFORCE's high variance in action[1]
2. **Smoothed curve (orange):** Moving average (window = 20 episodes)
 - Much smoother, reveals underlying trend
 - Computed as: $\text{smooth}[i] = \text{mean}(r[i - 10 : i + 10])$
 - Reduces noise while showing real progress
 - Makes the instability/recovery pattern visible[1]

Why moving average helps:

The jagged raw curve makes it hard to see if the agent is actually learning. Moving average filters out single-episode noise while revealing true learning progress.

Key Observation:

"The large spikes and drops illustrate the high variance of REINFORCE—individual unlucky episodes can dramatically reduce performance before recovery."[1]

This shows REINFORCE's fundamental problem: one bad episode can undo progress. PPO's clipping mechanism prevents this.

Slide 17: Key Findings & Insights

Five major conclusions from all experiments:

1. PPO Convergence Speed

- 40-50% faster than REINFORCE
- Reaches maximum reward in ~150 episodes vs REINFORCE's ~250
- Statistical significance: consistent across seeds[1]

2. Stability Advantage

- PPO shows significantly lower variance (narrower shaded bands)
- More reproducible across different random seeds
- Suitable for real-world deployment[1]

3. Gamma (γ) Optimization

- $\gamma = 0.99$ provides best balance
- Myopic (0.90): undershoots and plateaus
- Greedy (0.999): overshoots and delays learning
- Sweet spot enables both fast and stable learning[1]

4. Clip Range (ϵ) Impact

- $\epsilon = 0.2$ is optimal
- Too restrictive (0.1): learning too slow
- Too permissive (0.4): oscillatory, unstable
- Default value is well-tuned[1]

5. Learning Rate (α) Criticality

- $\alpha = 3 \times 10^{-4}$ is sweet spot
- $10\times$ smaller (3×10^{-5}): doesn't converge in available time
- $10\times$ larger (3×10^{-3}): oscillates and diverges
- Small differences cause dramatic performance changes[1]

Collective Insight:

"The existence of optimal hyperparameter settings means automated search has something meaningful to discover. The high sensitivity means naive manual tuning is insufficient."[1]

Slide 18: Practical Implications & Deployment

For Control Applications (robotics, autonomous systems)[1]:

- PPO should be default choice over REINFORCE
- Clipping mechanism provides safety margin for critical systems
- Sample efficiency matters in real-world where environment time is expensive

Hyperparameter Tuning Guidelines[1]:

- Always sweep γ across {0.95, 0.99, 0.999} for new domains
- Start with $\epsilon = 0.2$, adjust based on convergence speed
- Use learning rate schedules (decrease α over time) for longer training
- Run multiple seeds before deploying to production

Production Considerations[1]:

- PPO's stability is critical for autonomous systems
 - Monitor variance across seeds before deployment
 - A system that works once but fails randomly is undeployable
 - The shaded bands (uncertainty bands) should inform deployment decisions
-

Slide 19: Conclusions & Future Directions

Summary of Findings:

PPO demonstrates superior performance over REINFORCE on CartPole-v1 with:

- 40-50% faster convergence
- Lower variance and better reproducibility
- Careful hyperparameter selection is crucial ($\gamma = 0.99$, $\epsilon = 0.2$, $\alpha = 3 \times 10^{-4}$)[1]

The Core Thesis Proven:

Even on a "simple" benchmark like CartPole, hyperparameter choices have dramatic effects. Manual tuning is not sufficient; **Automated Reinforcement Learning (AutoRL) is necessary**. [1]

Future Research Directions[1]:

1. Complex Environments:

- Test on MuJoCo (continuous control) and Atari (visual)
- See if findings scale beyond CartPole

2. Automated Hyperparameter Tuning:

- Implement AutoRL (grid search, Bayesian optimization, population-based training)
- Meta-learn optimal hyperparameter distributions

3. Algorithm Expansion:

- Compare with A3C, TRPO, SAC for comprehensive benchmarking
- See if other algorithms show similar sensitivity patterns

4. Robustness Testing:

- Add adversarial perturbations
- Test with environmental noise
- Stress-test against distribution shift

5. Theoretical Analysis:

- Derive convergence guarantees for PPO on CartPole
 - Analyze sample complexity bounds
 - Understand why these specific hyperparameters work
-

Slide 20: Thank You & Contact

Project Conclusion: AutoRL is not a luxury but a necessity for practical RL.

Presenter Information[1]:

- Purvika Panwar
- Hochschule Schmalkalden
- Automated Reinforcement Learning Seminar
- Matriculation No. 319301
- January 2026

4. Key Formulas and Mathematical Foundations

Formula 1: Expected Discounted Return

$$J(\theta) = E \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Interpretation:

- The fundamental objective in reinforcement learning
- What every RL algorithm tries to maximize
- γ (discount factor) determines time preference[1]

Formula 2: Discounted Return (backward from time t)

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Interpretation:

- Total reward from time t onward
- Each reward is weighted by γ^k (older rewards are discounted)
- Used to compute targets for learning algorithms[1]

Formula 3: Mean and Standard Deviation

$$\bar{R}^e = \frac{1}{K} \sum_{k=1}^K R_k^e$$

$$\sigma^e = \sqrt{\frac{1}{K} \sum_{k=1}^K (R_k^e - \bar{R}^e)^2}$$

Interpretation:

- Mean ($\bar{R}^e$): Average reward across K random seeds
- Standard deviation (σ^e): Spread/variability across seeds
- Shaded regions in plots = $[\bar{R}^e - \sigma^e, \bar{R}^e + \sigma^e]$

- Used to quantify reproducibility[1]

Formula 4: REINFORCE Update Rule

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \nabla \log \pi_{\theta}(a_t|s_t) G_t$$

Interpretation:

- Empirical (Monte Carlo) estimate of the policy gradient
- Update weights to increase probability of actions leading to high returns
- Unbiased but high variance (each trajectory affects update)[1]

Formula 5: PPO Clipped Objective

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Where:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ (probability ratio)
- $\hat{A}_t$ (advantage estimate)
- $\text{clip}(x, a, b) = \min(\max(x, a), b)$ (clipping function)[1]

Interpretation:

- Takes minimum of two terms (unclipped and clipped)
- Acts as safety mechanism preventing large policy changes
- Clipping range ϵ controls maximum allowed change[1]

Formula 6: Probability Ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

Interpretation:

- Ratio of new policy to old policy probability
- $r_t = 1$: no policy change
- $r_t > 1$: policy increased probability of action
- $r_t < 1$: policy decreased probability of action
- PPO keeps $r_t \in [1 - \epsilon, 1 + \epsilon]$ [1]

Formula 7: Advantage Estimate

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

Where:

- $Q(s_t, a_t)$ = Expected return if taking action a_t in state s_t
- $V(s_t)$ = Expected return for state s_t regardless of action

Interpretation:

- How much better is this specific action versus average action?
- Positive: above-average action (should increase probability)
- Negative: below-average action (should decrease probability)
- Reduces variance in gradient estimates[1]

Formula 8: Clipping Function

$$\text{clip}(x, a, b) = \begin{cases} a & \text{if } x < a \\ x & \text{if } a \leq x \leq b \\ b & \text{if } x > b \end{cases}$$

Interpretation:

- Restricts value to interval [a, b]
 - If x outside range, clamp to boundary
 - In PPO: $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ keeps probability ratio in check[1]
-

5. Understanding Graphs and Visualizations

Common Graph Elements

X-axis (Horizontal):

- Episode number (usually 0-300 for CartPole)
- Each episode is one complete game (pole falls or 500 timesteps reached)

Y-axis (Vertical):

- Episode reward (usually 0-500)
- Total points earned in that episode
- Maximum: 500 (perfectly balanced entire episode)
- Minimum: ~20-25 (random untrained agent)

Solid Line:

- Mean (average) performance across multiple seeds
- Shows typical performance you can expect

Shaded Region (Confidence Band):

- ± 1 standard deviation around the mean
- Shows variability and uncertainty
- Narrow band = reproducible, consistent
- Wide band = sensitive to random seed[1]

Interpreting Shaded Bands

What narrow bands mean:

- All seeds reached similar performance
- If you run this algorithm again with a different seed, expect similar results
- Algorithm is stable and reliable
- Good for production deployment[1]

What wide bands mean:

- Different seeds gave very different results
- Some seeds succeeded, others failed
- If you run this algorithm again, outcome is unpredictable
- Algorithm is unstable and unreliable
- Need better hyperparameter tuning or stronger algorithm[1]

Why std deviation specifically?

Standard deviation is the "natural" measure of spread for Gaussian (normal) distributions. When averaging over multiple seeds, results tend to be approximately Gaussian, so std is the right uncertainty measure.[1]

Reading Multi-Curve Graphs

Comparing three curves (e.g., different γ values):

1. Compare final performance (where curves end)
2. Compare convergence speed (how quickly they rise)
3. Compare stability (band width throughout)
4. Identify plateau points (where learning stops)

Example: Gamma sensitivity graph shows $\gamma = 0.99$ is best because:

- Reaches highest final reward (500)
- Gets there fastest (~150 episodes)
- Has narrowest band (most reproducible)[1]

Common Patterns in RL Learning Curves

Typical S-shaped curve:

- Phase 1 (flat): Agent learns nothing, random performance
- Phase 2 (steep): Agent discovers good behaviors, rapid improvement
- Phase 3 (plateau): Agent approaches optimum, slower improvements[1]

Jagged vs. smooth:

- Jagged (REINFORCE): High variance, unstable algorithm
- Smooth (PPO): Low variance, stable algorithm[1]

Wide band that narrows:

- Indicates algorithm learns to be reproducible
- Different seeds diverge initially, then converge[1]

6. Why This Project Matters: Practical Takeaways

The Core Problem Solved

Before this project: "Why doesn't my RL code work? Did I implement it wrong?"

After this project: "RL algorithms are inherently fragile. Success depends heavily on hyperparameter tuning, not just algorithm choice."[1]

Key Lesson 1: PPO > REINFORCE in Practice

- **REINFORCE**: Simple, theoretically elegant, but too unstable for production
- **PPO**: Slightly more complex, but dramatically more stable and practical
- **Real-world implication**: Use PPO as your starting algorithm[1]

Key Lesson 2: Hyperparameters Matter Enormously

Changing γ from 0.99 to 0.90 changes:

- Success rate from 100% to 0%
- Same algorithm, same code, different results

This isn't a minor implementation detail—it's fundamental to algorithm success.[1]

Key Lesson 3: You Need Multiple Seeds

Running RL once and reporting success is not honest. You must:

- Run multiple seeds (3-5 minimum)
- Report mean and standard deviation
- Visualize confidence intervals
- Only claim success if band is narrow[1]

Key Lesson 4: AutoRL is Essential

Manual hyperparameter tuning doesn't scale:

- How many γ values should I try? (Infinite possibilities)
- What learning rate works for my domain? (Different per task)
- When should I stop tuning? (How do I know it's optimal?)[1]

Solution: Automated methods like Bayesian optimization or population-based training to systematically search the hyperparameter space.[1]

References

[1] Panwar, P. (2026). Automated Reinforcement Learning: Stability and Hyperparameter Sensitivity on CartPole-v1. Automated Reinforcement Learning Seminar, Hochschule Schmalkalden.